

Computer Networks

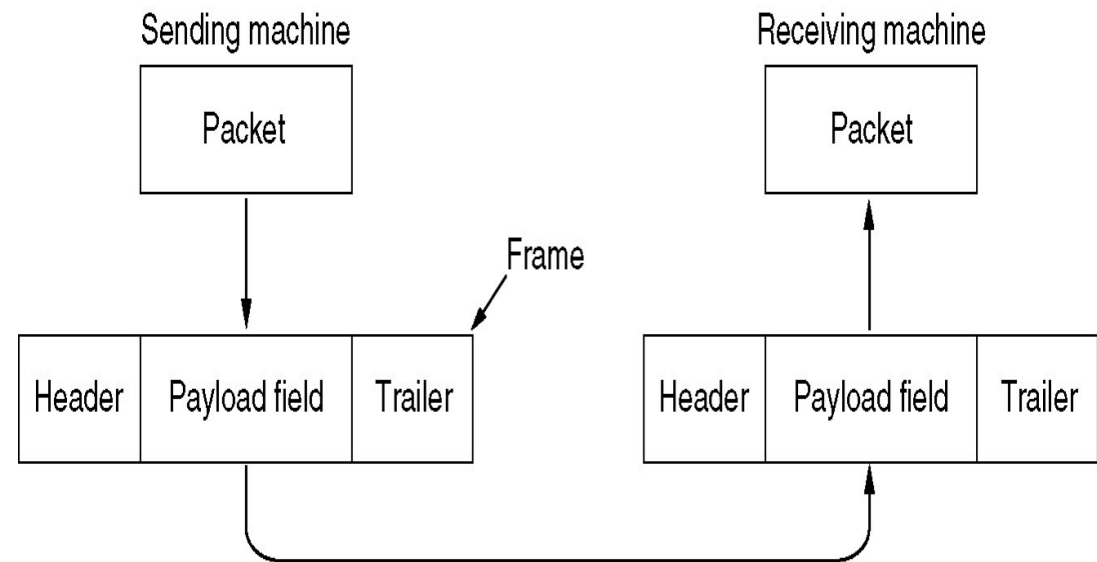
Lecture: Data Link Layer

Data Link Layer Design Issues

- Services Provided to the Network Layer
- Framing
- Error Control
- Flow Control

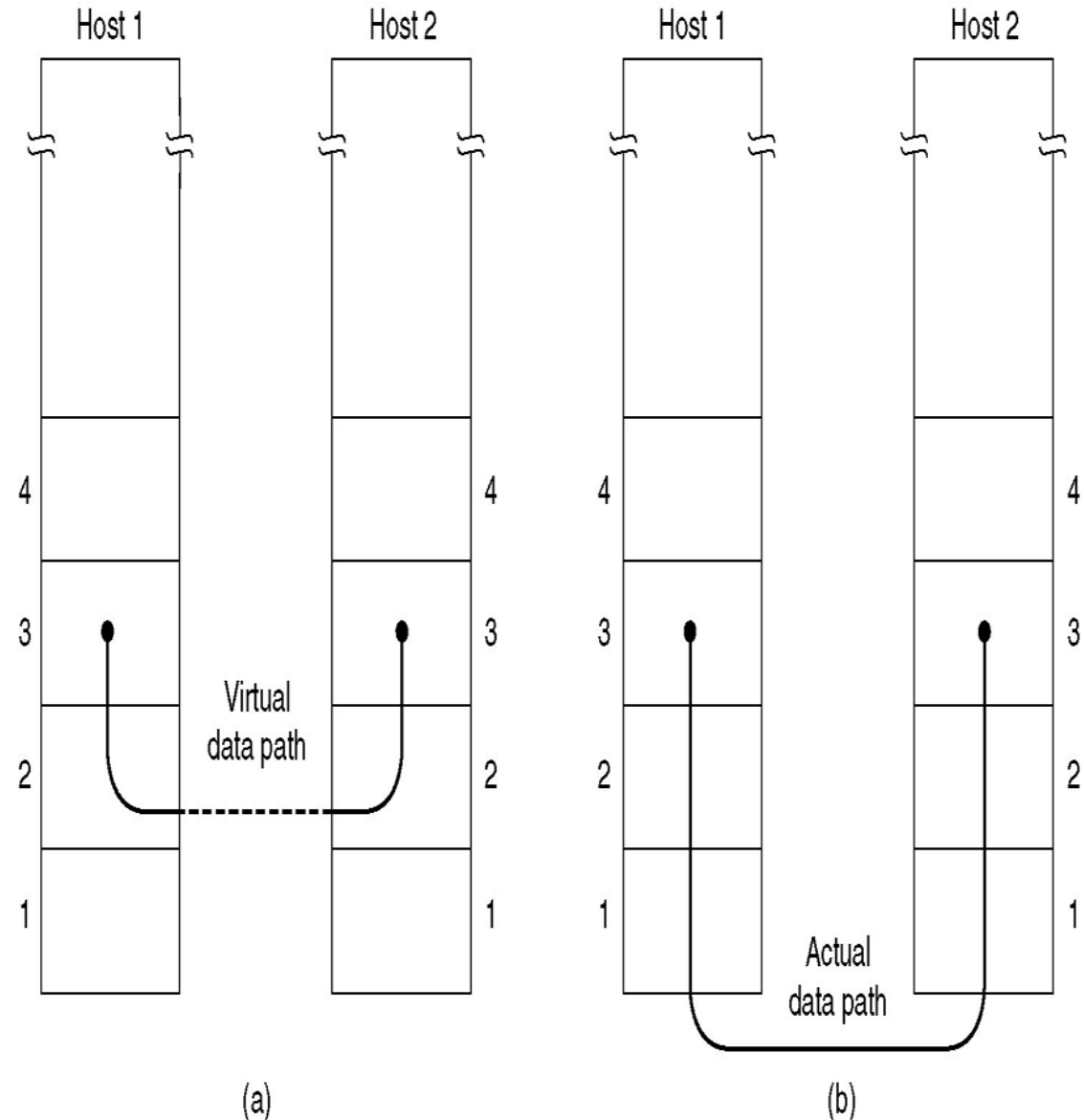
Functions of the Data Link Layer

- Provide service interface to the network layer
- Framing provides a way for a sender to transmit a set of bits that are meaningful to the receiver.
- Dealing with transmission errors
- Regulating data flow
 - Slow receivers not swamped by fast senders



Services Provided to Network Layer

- **Services Provided to the Network Layer** - Recall that lower layers provide services to higher layers that implement the actual physical path.
- The higher layer on the source communicates to the same layer on the destination via a *virtual* path created by the lower layers.



Types of services provided to the Network Layer

- Unacknowledged Connectionless service
- Acknowledged Connectionless service
- Acknowledged Connection-Oriented service

Unacknowledged Connectionless service

- Source machine send independent frames to the destination machine without having the destination machine acknowledge them
- No logical connection is established beforehand and released afterward
- If a frame is lost due to noise on the line, no attempt is made to detect the loss or recover from it in the data link layer.
- Appropriate for voice, where delay is worse than bad data.

Acknowledged Connectionless service

- There is no logical connections between sender and receiver
- Each frame sent is individually acknowledged. Hence the sender knows whether the frame has arrived correctly or not.
- If it is not arrived within a specified time interval, it can be send again.
- This service is useful over unreliable channels such as wireless system.
- Adding ack in the DLL rather than in the NL is just an optimization and not a requirement.

Acknowledged Connection-oriented service

- Most reliable
- Source and destination machine establishes a connection before any data are sent
- Guaranteed service –
 - Each frame sent is indeed received
 - Each frame is received exactly once
 - Frames are received in order

Framing

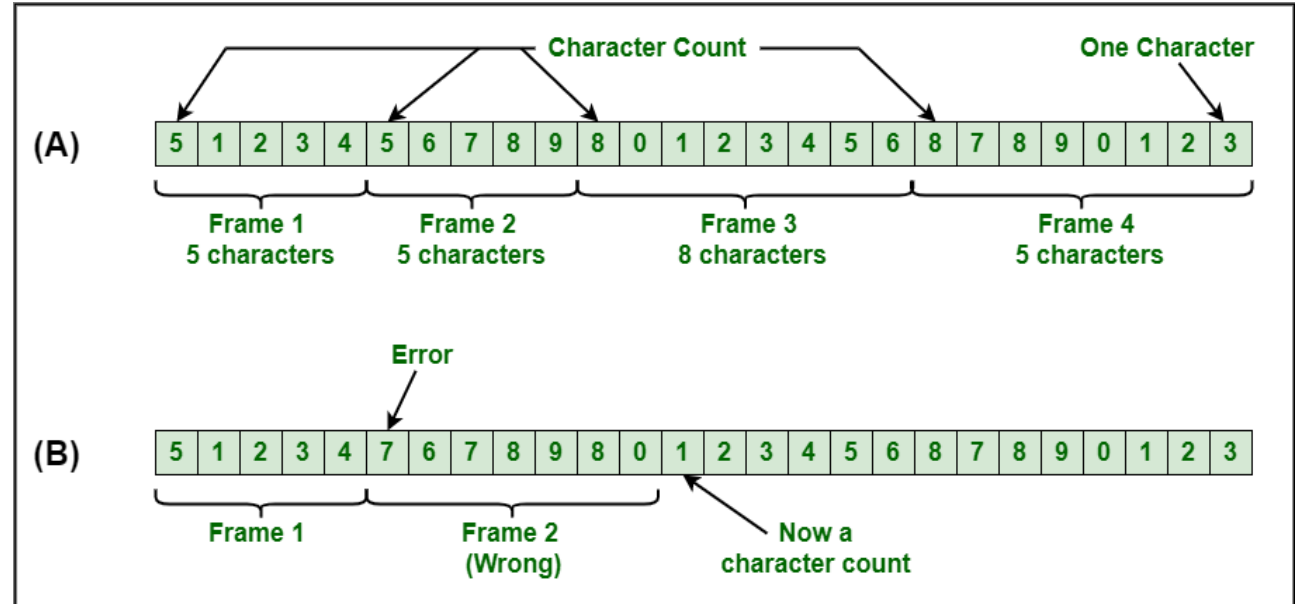
- Framing is a function of the data link layer.
- It provides a way for a sender to transmit a set of bits that are meaningful to the receiver.
- Ethernet, token ring, frame relay, and other data link layer technologies have their own frame structures.
- Frames have headers that contain information such as error-checking codes.

There are four different types of framing, each of which provides a way for the sender to tell the receiver where the block of data begins and ends:

- Byte Count
- Flag bytes with byte stuffing
- Flag bytes with bit stuffing
- Physical Layer Violation

Byte Count

- This framing method uses a field in the header to specify the number of bytes in the frame.
- When the data link layer at the destination sees the byte count, it knows how many bytes follow and hence where the end of the frame is.
- What if the count is garbled..?



A Character Stream

- (A) Without Errors
- (B) With one Error

Problem with byte count

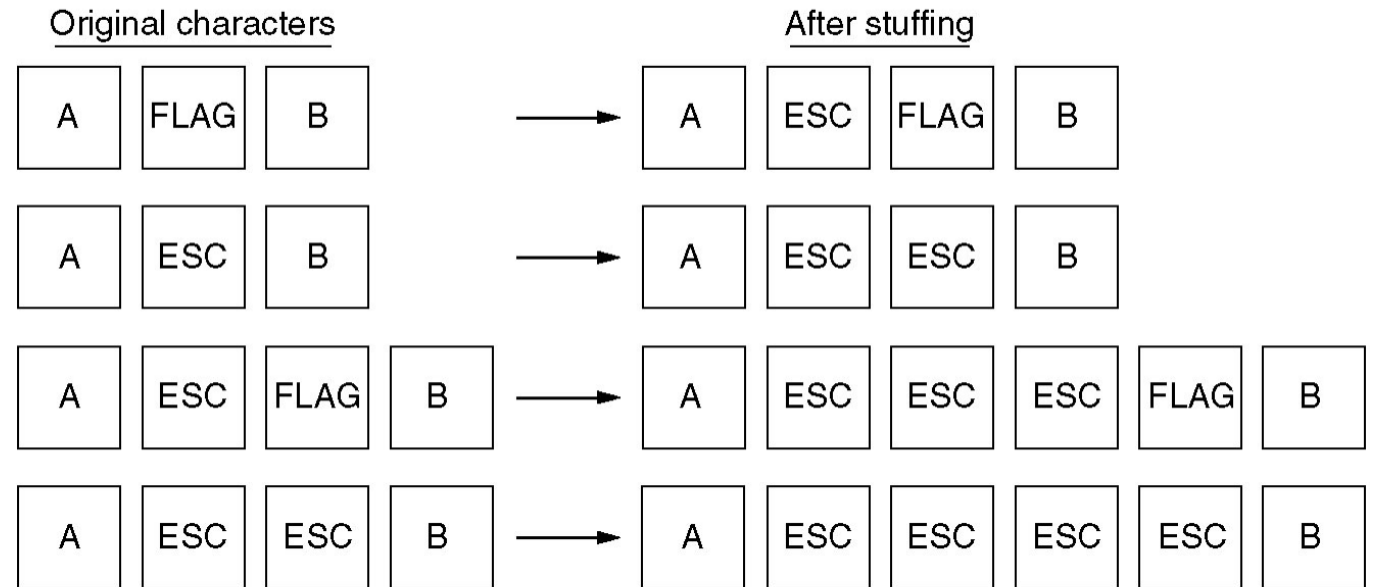
- Even if with checksum, the receiver knows that the frame is bad there is no way to tell where the next frame starts.
- Asking for retransmission doesn't help either because the start of the retransmitted frame is not known.
- Rarely used method

Framing with byte stuffing

- Each frame starts and end with special bytes called flag byte.
- Two consecutive flag byte indicates the end of one frame and start of the other frame.
- If the receiver loses synchronization it can just search for two flag bytes to find the end of the current frame and start of the next frame



(a)



(b)

Problem in byte stuffing

- Flag byte can occur in data
- To solve this, to solve this insert a special character called ESC(escape byte) at the sender's data link layer just before each "accidental" flag byte in the data.
- Hence the framing flag byte can be distinguished from one in the data by the absence or presence of ESC before it.
- The data link layer at the receiving end removes the escape bytes before giving it to the network layer.
- This technique is known as *byte stuffing*
- The data become so bulky

Framing with bit stuffing

- Bit stuffing is also known as bit-oriented framing or bit-oriented approach.
- Framing done at the bit level.
- Frames can contain an arbitrary number of bits made up of units of any size
- Each frame begins with a special bit pattern, 01111110 or 0x7E in hexadecimal
- Whenever the sender's data link layer encounters five consecutive 1's in the data, it automatically stuffs a 0 bit into the outgoing bit stream.

- When the receiver sees five consecutive incoming 1 by, followed by a 0 bit, it automatically destuffs the 0 bit
- The boundary between two frames can be recognized by the flag pattern.
- If the receiver loses track of where it is, scan the input for flag sequence, since they only occur at the frame boundaries and never within the data
- The data become so bulky

Assume we send a frame of 2 8-bit characters:

01111111 01111101
char 1 char 2

On the line we will send:

start of frame end of frame
01111110 011111011 011111001 01111110
char 1 char 2

sender rule is - if 5 1s in data add (stuff) a zero

Receiver strips start & end of frame and un-stuffs:

01111110 011111-11 011111-01 01111110
char 1 char 2

receiver rule is - if a zero occurs after 5 1s remove it

Bit stuffing

(a) The original data.

(b) The data as they appear on the line.

(c) The data as they are stored in receiver's memory after destuffing.

(a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0

Stuffed bits

(c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Physical Layer Coding Violations

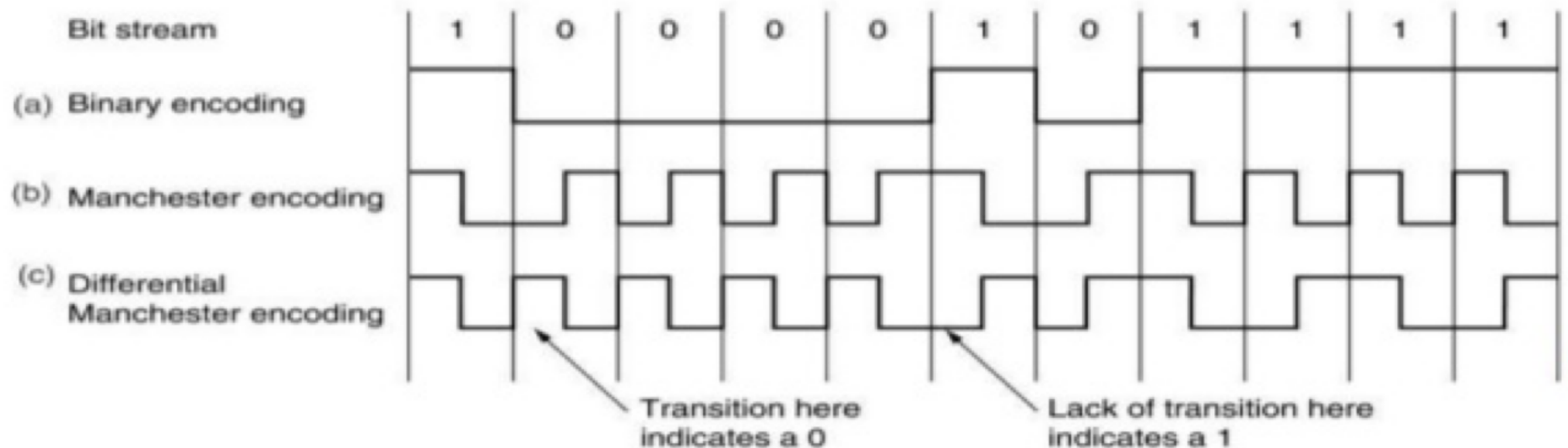
- Encoding on physical medium
- Framing by invalid physical code
 - Use some code that does not represent 0 nor 1
- Example
 - Manchester encoding in 802 LAN standard
 - 4B/5B line code
- Combination of approaches is possible
 - E.g., character count + one other method
 - Preamble in Ethernet followed by length

Physical layer coding violations

- This method of framing is only applicable to
 - networks in which the encoding on the physical
 - medium contains some redundancy.
 - For example: some LANs use Manchester
 - Encoding where a 1 bit is a high-low pair and a 0 bit is a low-high pair. And the combinations high-high and low-low are not used for data but are used for delimiting frames in some protocols.
-

PHYSICAL LAYER CODING VIOLATIONS

- This Framing Method is used only in those networks in which Encoding on the Physical Medium contains some redundancy.
- Some **LANs** encode each bit of data by using two Physical Bits i.e. Manchester coding is Used. Here, Bit **1** is encoded into high-low(10) pair and Bit **0** is encoded into low-high(01) pair.
- The scheme means that every data bit has a transition in the middle, making it easy for the receiver to locate the bit boundaries. The combinations high-high and low-low are not used for data but are used for delimiting frames in some protocols.



Error Control

- Positive and Negative feedback
- Timers : what happens when a frame completely vanishes : receiver neither sends a +ack nor -ack ... then timer comes to help.
 - It may result in a frame being sent more than once and received more than once :
 - solution : assign sequence numbers to frames

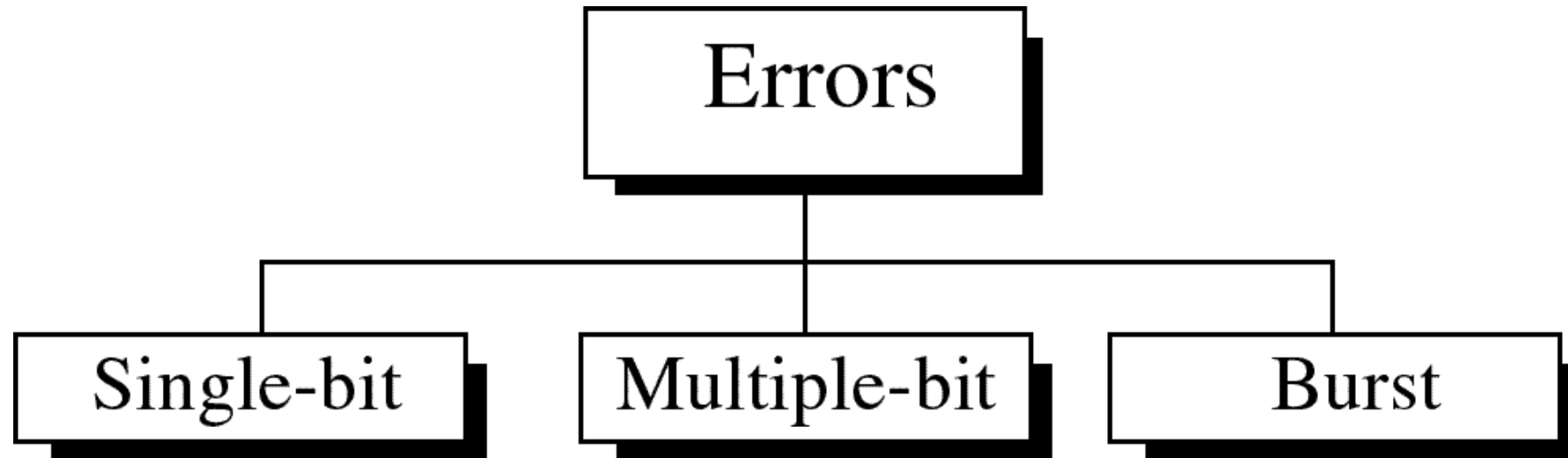
Error Detection and Correction

- In some cases it is sufficient to detect an error and in some, it requires the errors to be corrected also. For example
 - On a reliable medium : ED is sufficient where the error rate is low and asking for retransmission after ED would work efficiently
 - In contrast, on an unreliable medium : Retransmission after ED may result in another error and still another and so on. Hence EC is desirable.

Error detecting code

- Parity
- Checksum
- Polynomial code or CRC(Cyclic Redundancy Check)

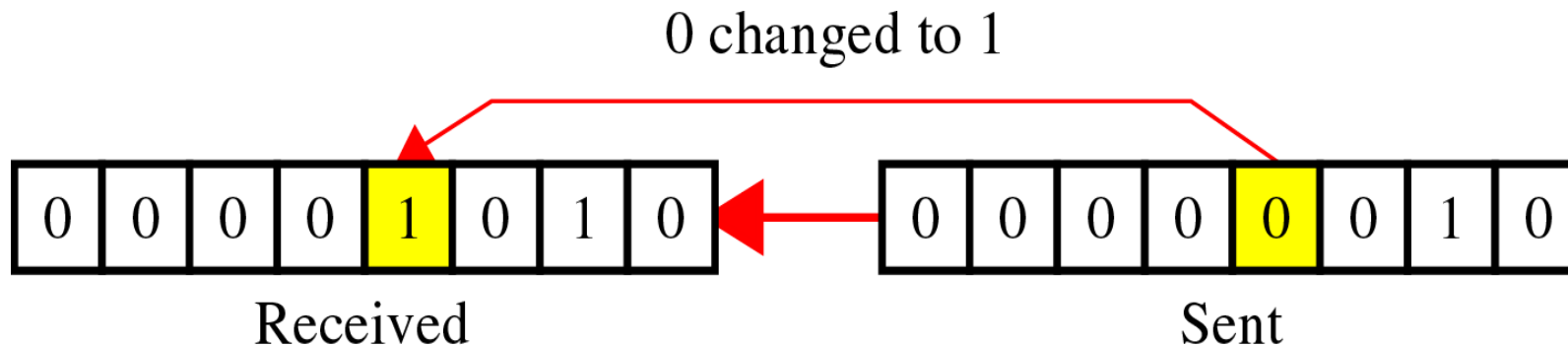
Type of Errors



Type of Errors

- Single-Bit

~ Error
Is when only one bit in the data unit has changed

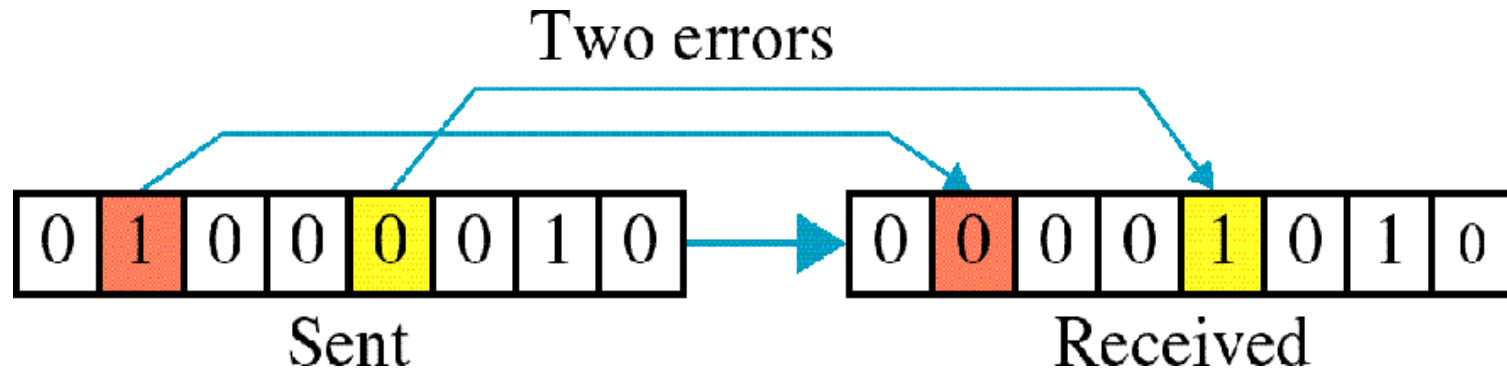


Single bit errors are the **least likely** type of errors in serial data transmission because the noise must have a very short duration which is very rare. However this kind of errors can happen in parallel transmission.

Type of Errors

- Multiple-Bit Error

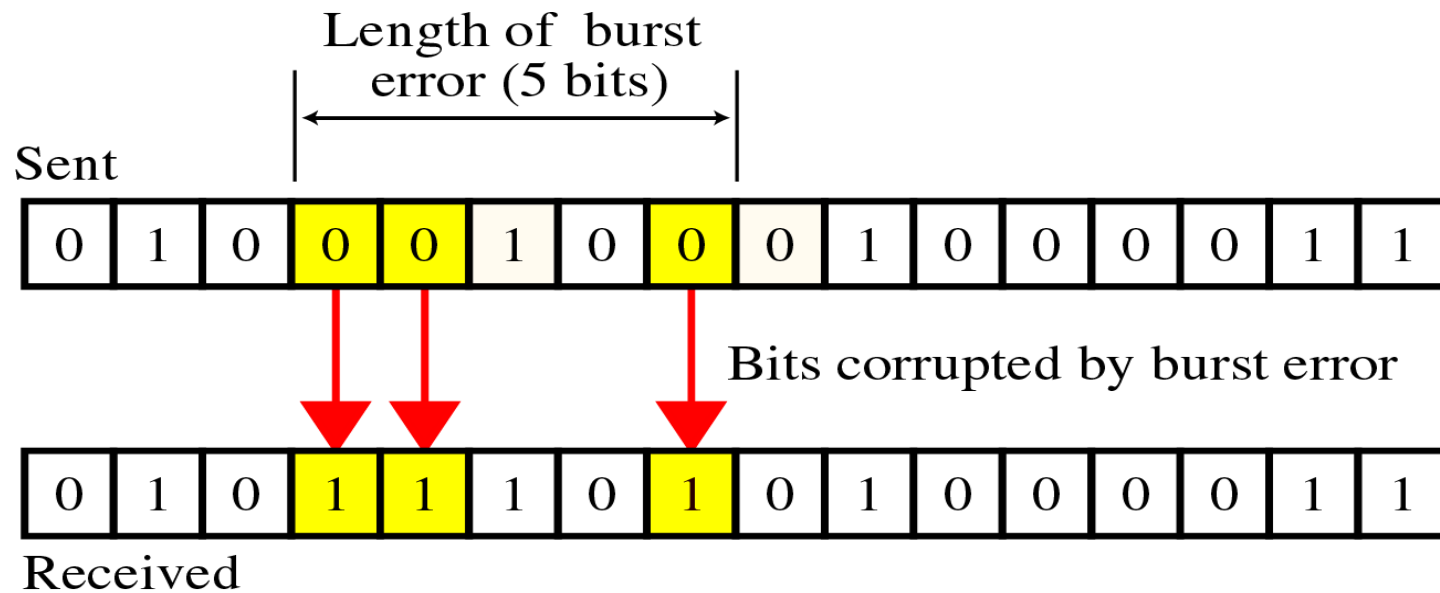
~ is when two or more nonconsecutive bits in the data unit have changed



Type of Errors

□ Burst Error

~ means that 2 or more consecutive bits in the data unit have changed



The term **burst error** means that two or more bits in the data unit have changed from 1 to 0 or from 0 to 1.

Burst errors does not necessarily mean that the errors occur in consecutive bits, the length of the burst is measured from the first corrupted bit to the last corrupted bit. Some bits in between may not have been corrupted.

- ★ **Burst error is most likely to happen in serial transmission** since the duration of noise is normally longer than the duration of a bit.
- ★ The number of bits affected depends on the data rate and duration of noise.

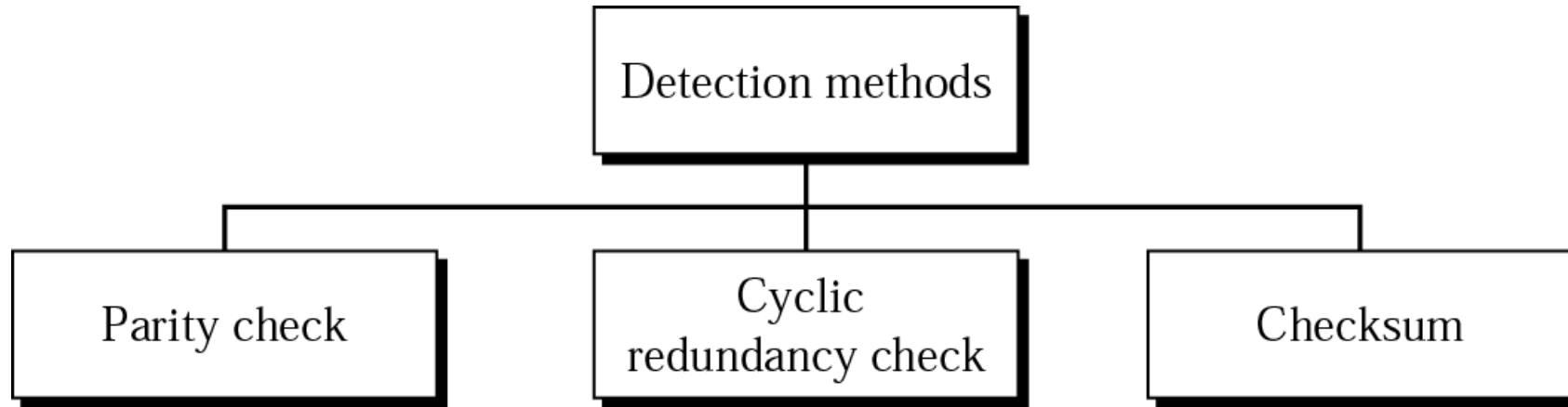
Error detection

Error detection

Error detection means to decide whether the received data is correct or not without having a copy of the original message.

Error detection **uses the concept of redundancy, which means** adding extra bits for detecting errors at the destination.

□ Detection methods



Parity

- Value of parity bit is such that character has even (even parity) or odd (odd parity) number of ones
- Even number of bit errors goes undetected
- Simple Parity
- Longitudinal Redundancy Check/Vertical Redundancy Check (LRC/VRC)

Data							Parity
1	0	0	1	1	1	0	0
0	0	1	1	1	0	0	1
1	1	1	0	1	1	1	0

Data							VRC
1	0	0	1	1	1	0	0
0	0	1	1	1	0	0	1
1	1	1	0	1	1	1	0
1	1	1	0	1	1	1	0
0	0	1	1	1	0	0	1
0	0	1	0	1	0	1	1
0	0	1	1	1	0	0	1
1	0	0	0	1	1	1	0

LRC

Parity Examples - Using Even Parity

- Even Parity of 1's

- LRC/VRC

Parity Checks

- Simple parity - If performing even parity, add a parity bit such that an even number of 1s are maintained.
- If performing odd parity, add a parity bit such that an odd number of 1s are maintained.
- For example, if the character 1001010 is to be sent, using even parity, a parity bit = 1 would be added to the character.
- If the character 1001011 is to be sent, using even parity, a parity bit = 0 would be added to the character.

Parity Checks

- What happens if the character 10010101 (parity bit is the last bit) and the first two 0s accidentally become two 1s?
- Thus, the following character is received: 11110101.
- Will there be a parity error?
- Problem: Simple parity only detects odd numbers of bits in error (50%)

Parity Checks

- Longitudinal parity adds a parity bit to each character then adds a row of parity bits after a block of characters.
- The row of parity bits is actually a parity bit for each “column” of characters.
- The row parity bits plus the column parity bits add a great amount of redundancy to a block of characters.

Table 6-2 *Simple example of longitudinal parity*

	Data						Parity	
Row 1	1	1	0	1	0	1	1	1
Row 2	1	1	1	1	1	1	1	1
Row 3	0	1	0	1	0	1	0	1
Row 4	0	0	1	1	0	0	1	1
Parity Row	0	1	0	0	1	1	1	0

Table 6-3 *The second and third bits in Rows 1 and 2 have errors, but longitudinal parity does not detect the error*

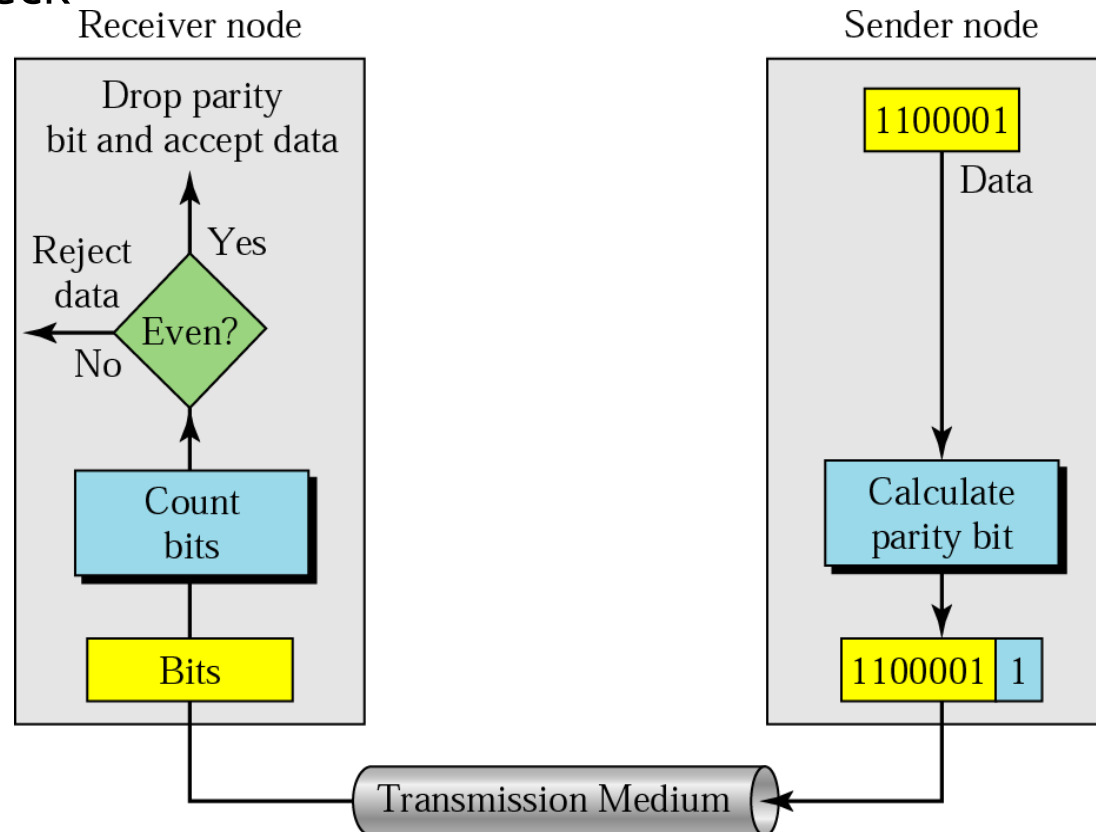
		Data			Parity			
Row 1	1	1 0	0 1	1	0	1	1	1
Row 2	1	1 0	1 0	1	1	1	1	1
Row 3	0	1	0	1	0	1	0	1
Row 4	0	0	1	1	0	0	1	1
Parity Row	0	1	0	0	1	1	1	0

Parity Checks

- Both simple parity and longitudinal parity do not catch all errors.
- Simple parity only catches odd numbers of bit errors (50% of all errors)
- Longitudinal parity is better at catching errors but requires too many check bits added to a block of data.
- As such, these methods are not that often used. However, a parity bit exists in 1 byte of data.
- *We need a better error detection method. What about cyclic redundancy checksum?*

□ Parity Check

- A parity bit is added to every data unit so that the total number of 1s(including the parity bit) becomes even for even-parity check or odd for odd-parity check
- Simple parity check



Detection

-examples

Example 1

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

11101110 11011110 11100100 11011000 11001001

Detection – examples

Now suppose the word world in Example 1 is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000
11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 4). The data are accepted.

Detection – examples

Now suppose the word world in Example 1 is corrupted during transmission.

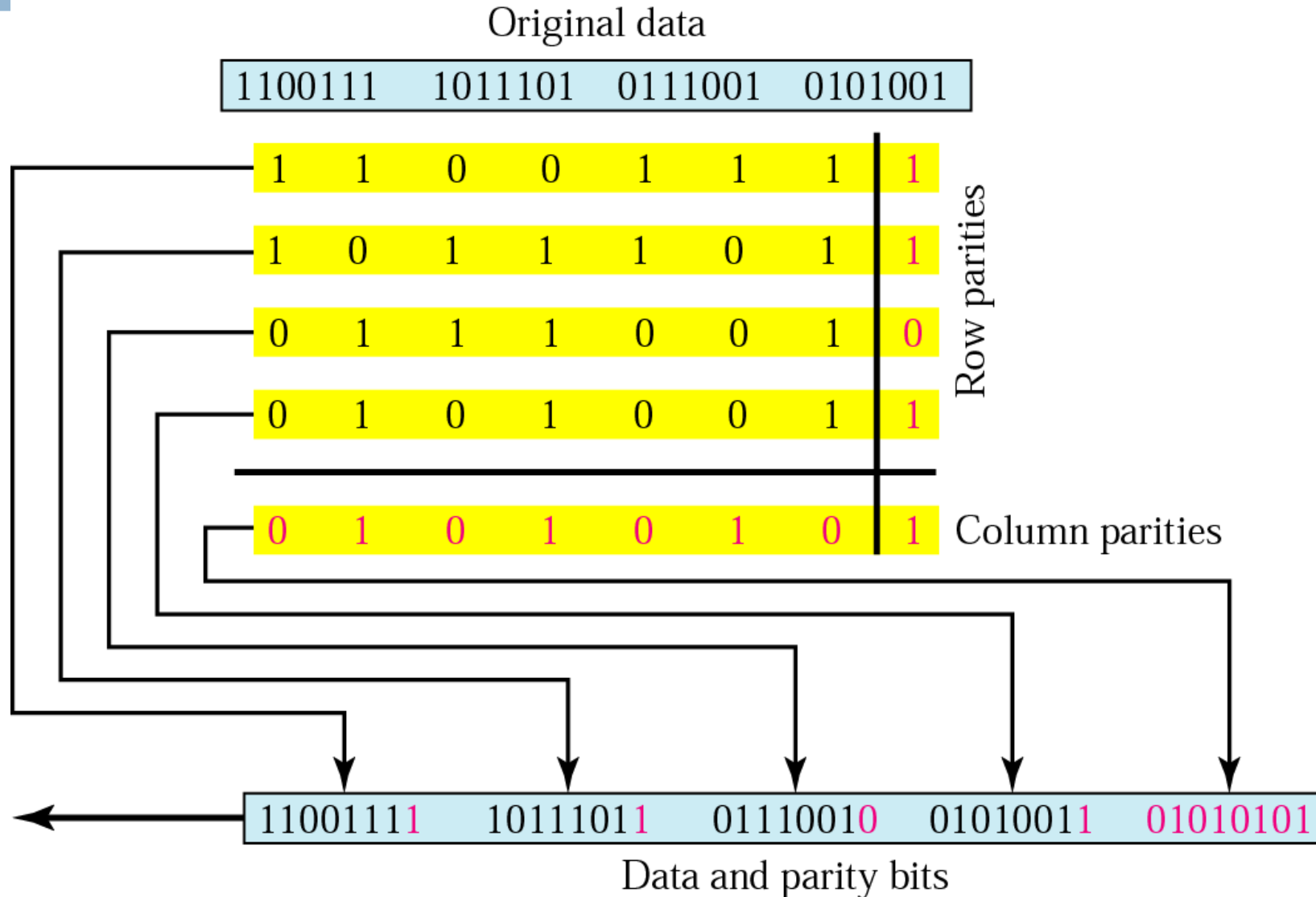
11111110 11011110 11101100 11011000
11001001

The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

- the simplest form of error detection is the use of a single **parity bit**.
- In an even parity scheme, the sender simply includes one additional bit and chooses its value such that the total number of 1s in the $d + 1$ bits (*the original information plus a parity bit*) is even. For odd parity schemes, the parity bit value is chosen such that there is an odd number of 1s.

- Receiver operation is also simple with a single parity bit. The receiver need only count the number of 1s in the received $d + 1$ bits. If an odd number of 1-valued bits are found with an even parity scheme, the receiver knows that at least one bit error has occurred. More precisely, it knows that some odd number of bit errors have occurred.

Two -Dimensional Parity Check



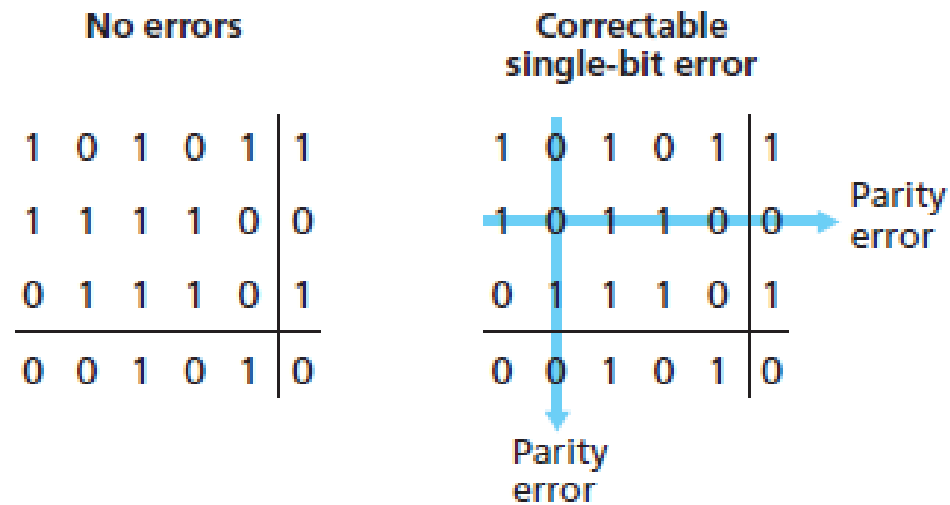
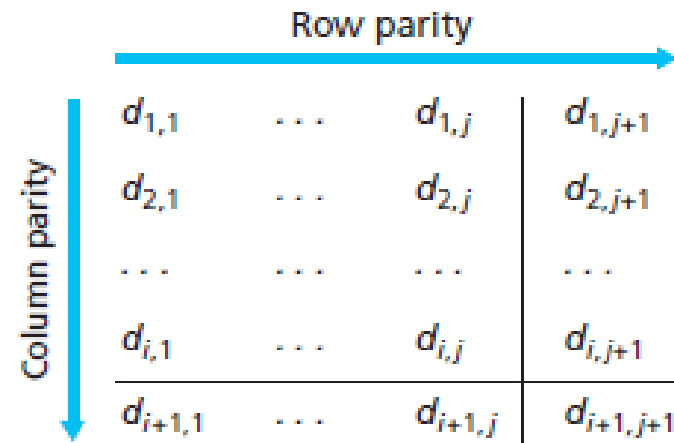


Figure 5.5 ♦ Two-dimensional even parity

- With this **two-dimensional parity scheme**, **the parity of both the column** and the row containing the flipped bit will be in error.
- The receiver can thus not only *detect the fact that a single bit error has occurred*, but can use *the column* and row indices of the column and row with parity errors to actually identify the bit that was corrupted and *correct that error*.
- The ability of the receiver to both detect and correct errors is known as **forward error correction (FEC)**. **These techniques are commonly used in audio storage and playback devices such as audio CDs.**

Detection -

Example

Suppose the following block is sent:

10101001 00111001 11011101 11100111 10101010

However, it is hit by a burst noise of length 8, and some bits are corrupted.

10100011 10001001 11011101 11100111 10101010

When the receiver checks the parity bits, some of the bits do not follow the even-parity rule and the whole block is discarded.

10100011 10001001 11011101 11100111 10101010

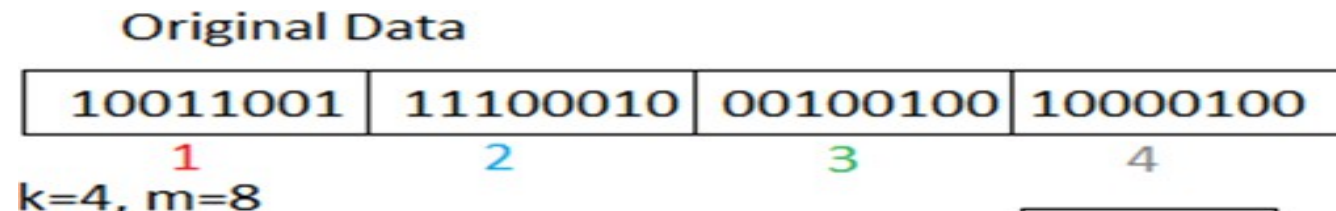
Detection

- Checksum

- ~ used by the higher layer protocols
- ~ is based on the concept of redundancy(VRC, LRC, CRC)

Checksum

- In checksum error detection scheme, the data is divided into **k segments** each of **m bits**.
- In the sender's end the segments are added using 1's complement arithmetic to get the sum. The sum is complemented to get the checksum.
- The checksum segment is sent along with the data segments.
- At the receiver's end, all received segments are added using 1's complement arithmetic to get the sum. The sum is complemented.
- If the result is zero, the received data is accepted; otherwise discarded.



Original Data

10011001	11100010	00100100	10000100
----------	----------	----------	----------

1

2

3

4

k=4, m=8

Sender

1 1 0 0 1 1 0 0 1
2 1 1 1 0 0 0 1 0

① 0 1 1 1 1 0 1 1
1

3 0 0 1 0 0 1 0 0
1 0 1 0 0 0 0 0

4 1 0 0 0 0 1 0 0
① 0 0 1 0 0 1 0 0
1

Sum: 0 0 1 0 0 1 0 1

Checksum: 1 1 0 1 1 0 1 0

Reciever

1 1 0 0 1 1 0 0 1
2 1 1 1 0 0 0 1 0

① 0 1 1 1 1 0 1 1
1

3 0 0 1 0 0 1 0 0
1 0 1 0 0 0 0 0

4 1 0 0 0 0 1 0 0
① 0 0 1 0 0 1 0 0
1

Sum: 1 1 1 1 1 1 1 1

Complement: 0 0 0 0 0 0 0 0

Conclusion: Accept Data

Cyclic Redundancy Check (CRC)

- ❖ Cyclic Redundancy Check (CRC) is an error detection technique to detect changes to raw data and is used widely in today's computer networks.
- ❖ CRC codes are also known as polynomial codes, since it is possible to view the bit string to be sent as a polynomial.

- Cyclic Redundancy Checksum (CRC)
- The CRC error detection method treats the packet of data to be transmitted as a large polynomial.
- The transmitter takes the message polynomial and using polynomial arithmetic, divides it by a given generating polynomial.
- The quotient is discarded but the remainder is “attached” to the end of the message (remainder (mod) arithmetic)

- Cyclic Redundancy Checksum
- The message (with the remainder) is transmitted to the receiver.
- The receiver divides the message and remainder by the same generating polynomial.
- If a remainder not equal to zero results, there was an error during transmission.
- If a remainder of zero results, there was no error during transmission.

Generator

- The sender and receiver must first agree on an *pattern, known as a generator, which we will denote as **G**.*
- If the generator is a polynomial, it must follow the following two properties:
 - a. It should not be divisible by x , because otherwise the last bit of the checksum would always be 0.
 - b. It should be divisible by x

The Technique

Suppose m is the number of bits of the CRC generator.

On Sender Side

➤ *Step 1*

- Append $(m-1)$ number of zeroes to the data on the Least Significant Bit (LSB) (rightmost) side.
- Divide it by the CRC Generator. (XOR operation is performed instead of Subtraction while dividing)

0	+	0	=>	0
1	+	0	=>	1
0	+	1	=>	1
1	+	1	=>	0

➤ *Step 2*

- The remainder from this division is added to the data generated earlier in Step 1.

This data is then sent to the destination

On Receiver Side

➤ *Step 1*

- Received data is again divided by the CRC generator.

➤ *Step 2*

- If remainder is zero No Error in Data
- If remainder is not zero Data was altered somewhere in the network.

CRC generation at sender side

Step 1 : Find the length of the divisor "L"

Step 2 : Append "L-1" bits to the original message

Step 3: Perform binary division (XOR)

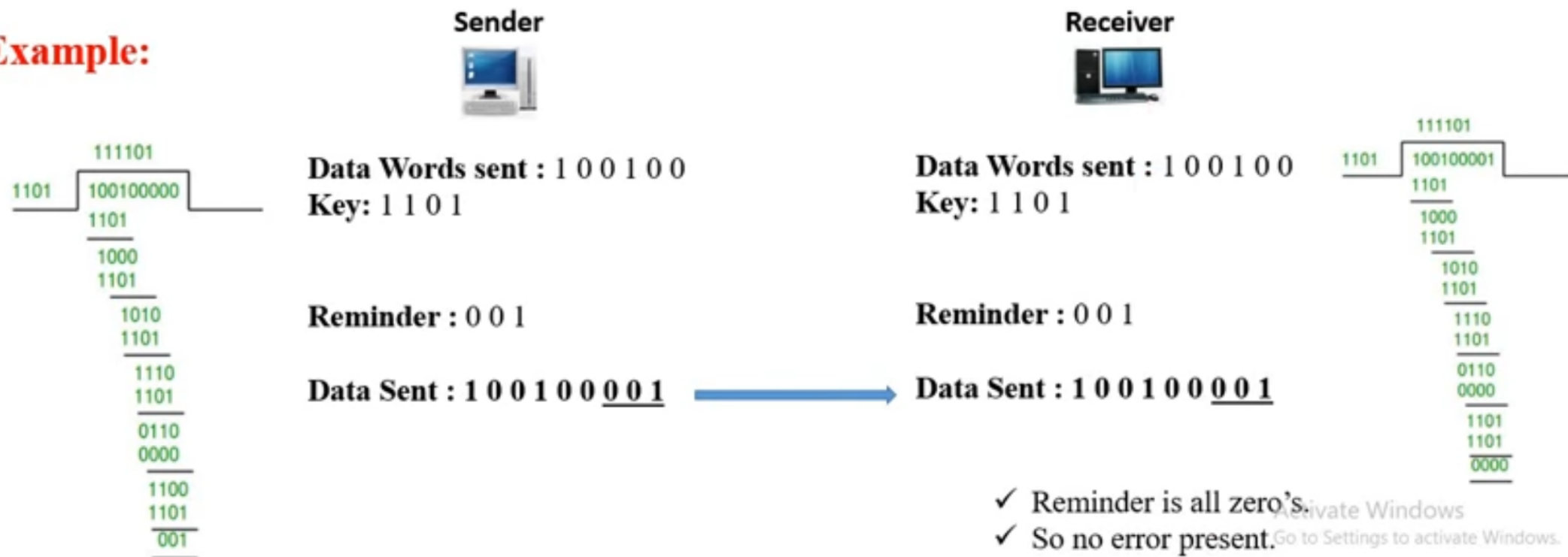
Operation

Step 4 : Remainder of the division = CRC

3. Cyclic Redundancy Check (CRC)

- CRC is a method of detecting accidental changes/errors in the communication channel.
- CRC uses Generator Polynomial which is available on both sender and receiver side.

Example:



Example

- In this example, we shall encode 14 bits of message.
- Generator is the polynomial $x^3 + x + 1$.
- The polynomial is written in binary as the coefficients ($1x^3 + 0x^2 + 1x + 1$).
- In this case, the coefficients are 1, 0, 1 and 1.
- Generator = 1011

- Start with the message to be encoded:
11010011101100
- We have $m = 4$, so we append 3 zeroes to the LSB side.
11010011101100**000**
- Now we divide It by 1011

- Remainder = 100
- Now Add 100 to our original dividend.

11010011101100**000**

+

100

11010011101100100

- If generator is $x^3 + 1$ and message to be sent is "CAT" . Find What Data is Sent. Further if 5th Bit is Changed show that the error can be detected.

C - 1000011

A - 1000001

T - 1010100

- Generator = 1001
- Append Three Zeroes
- Solve with the same steps

**Given are: $G(x) = x^3 + x + 1$ and $M(x) = x^3 + 1$.
Determine the message to be send ($T(x)$).**

Sender Side

$$\begin{array}{r}
 1001 \overline{) 101000000000} \\
 \underline{+1001} \\
 001100000000 \\
 \underline{+1001} \\
 0101000000 \\
 \underline{+1001} \\
 001100000 \\
 \underline{+1001} \\
 01010000 \\
 \underline{+1001} \\
 00110000 \\
 \underline{+1001} \\
 0011
 \end{array}$$

Message to be sent: $1010000\cancel{000}$
 $+011$

 $1010000\color{red}{011}$

Receiver Side

$$\begin{array}{r}
 1001 \overline{) 10100000011} \\
 \underline{+1001} \\
 00110000011 \\
 \underline{+1001} \\
 01010011 \\
 \underline{+1001} \\
 0011011 \\
 \underline{+1001} \\
 01001 \\
 \underline{+1001} \\
 0000
 \end{array}$$

**Zero means
data is received
without errors**

Original message:

1010000

Generator polynomial:

 x^3+1
$$1.x^3 + 0.x^2 + 0.x^1 + 1.x^0$$

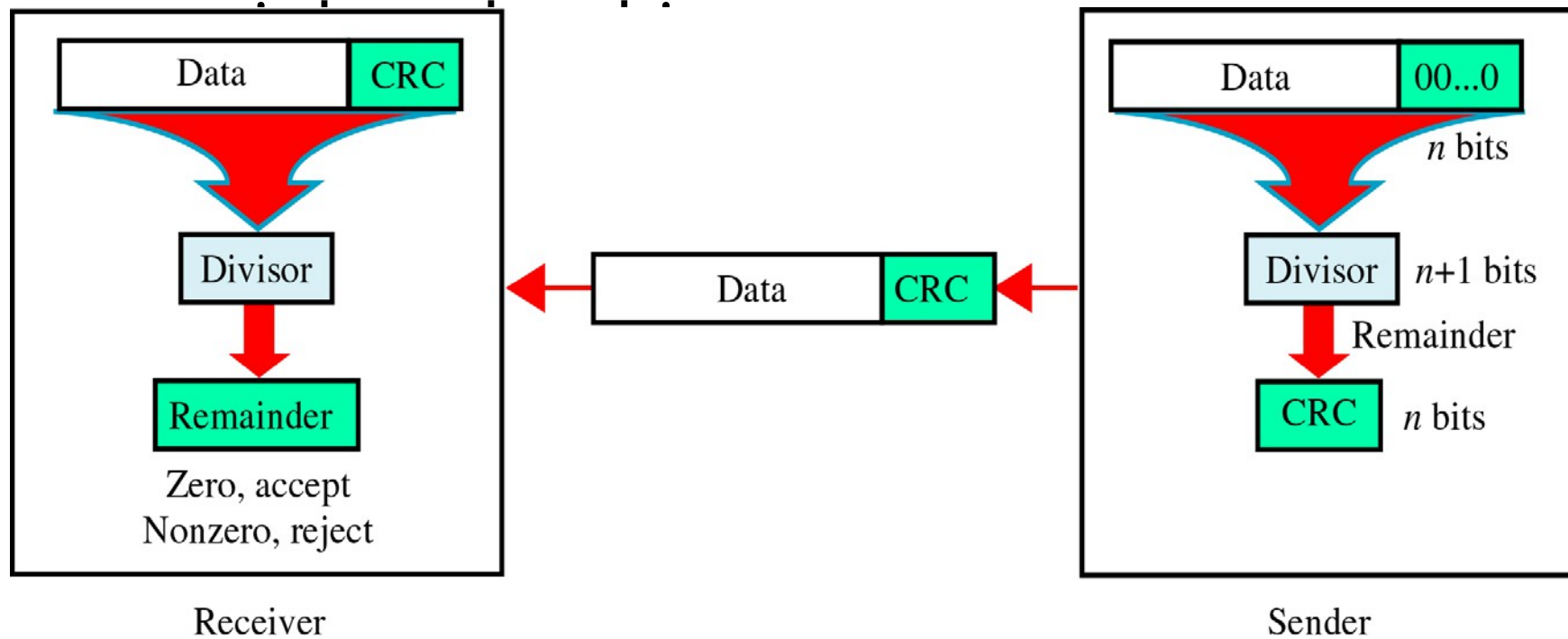
CRC generator:

1001

If CRC generator is of n -bit then append $n-1$ zeros to the end of original message

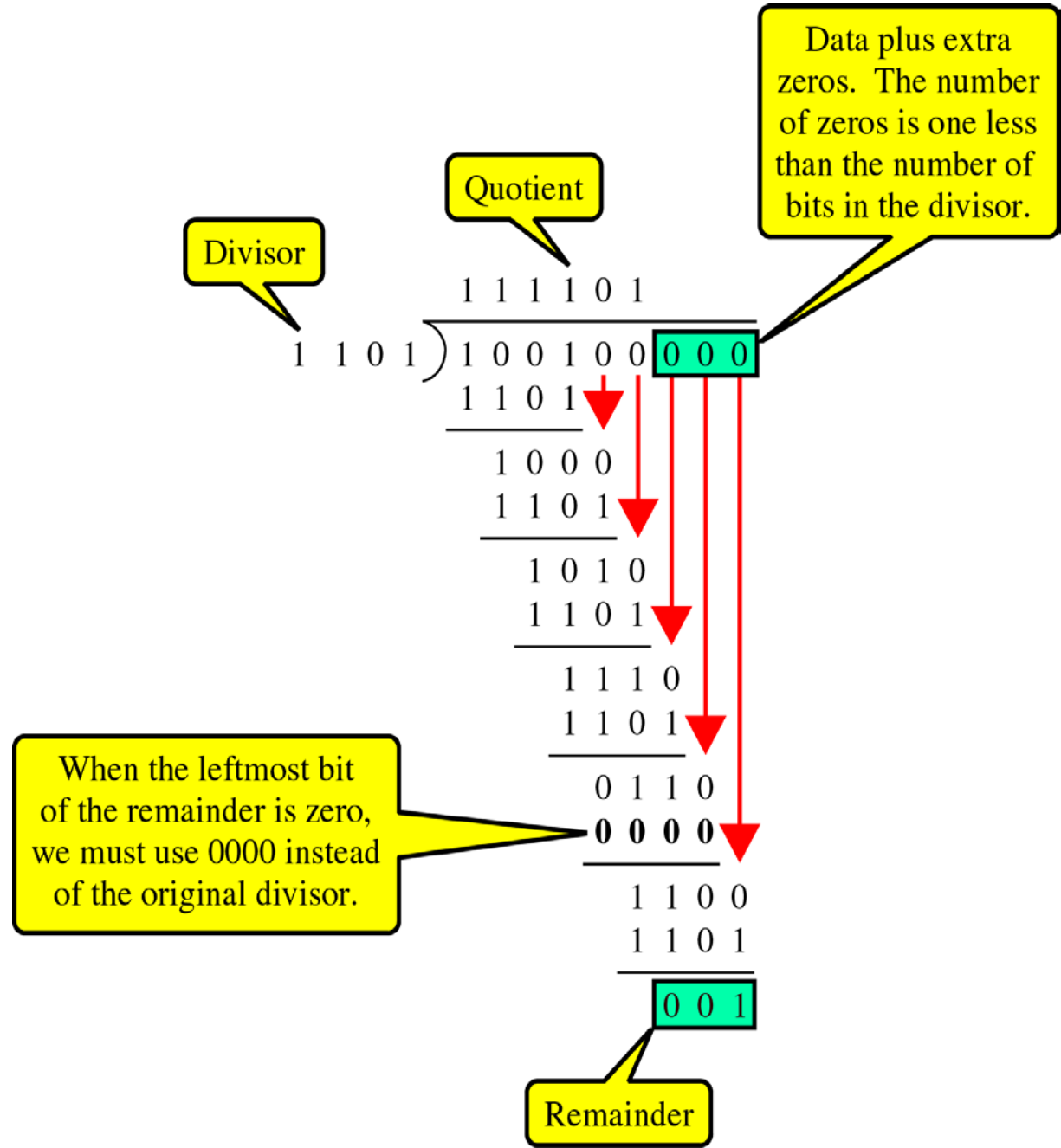
$\oplus \Rightarrow$ X-OR operation

□ CRC(Cyclic Redundancy Check)



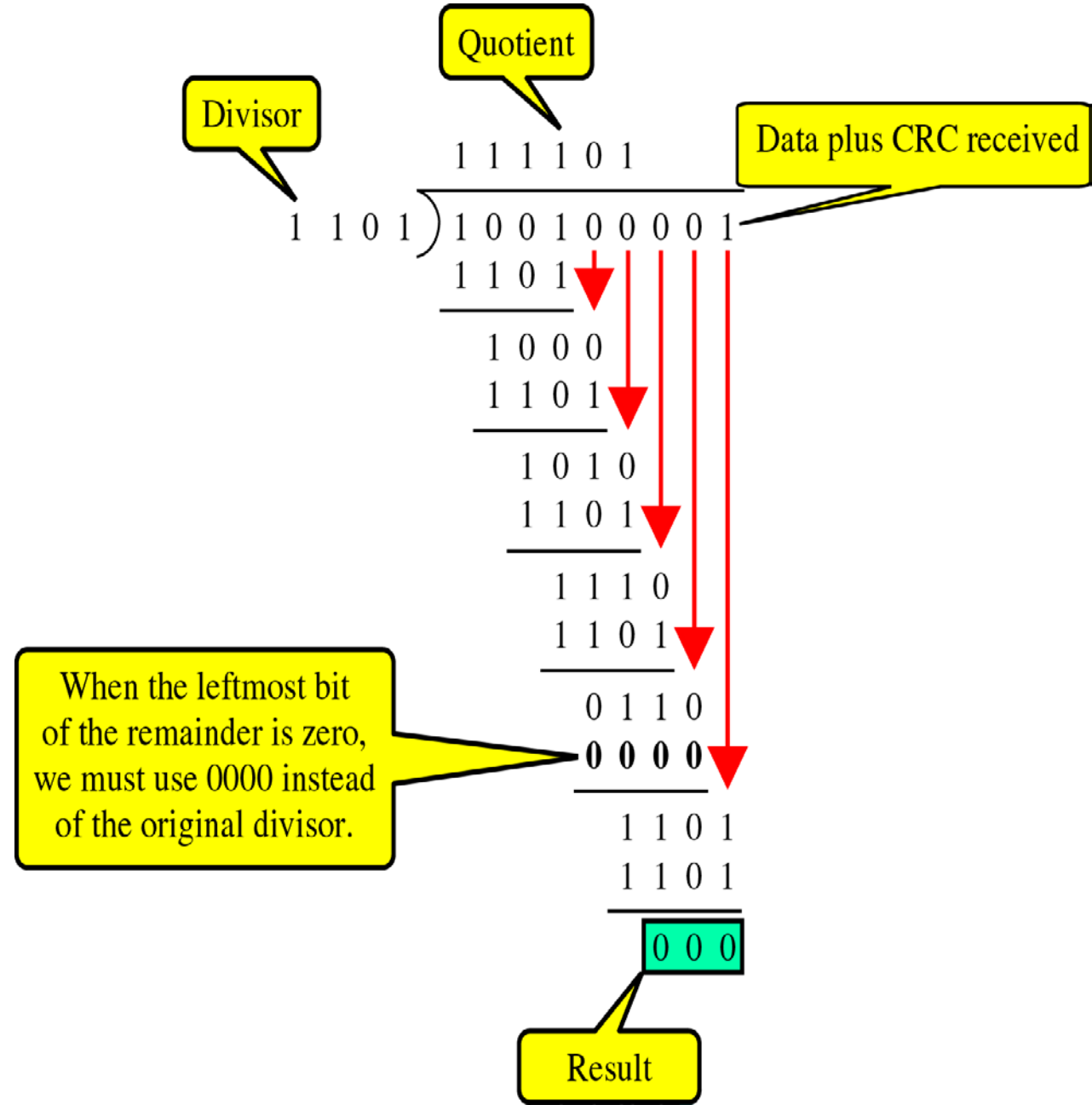
□ CRC generator

~ uses modular-2 division.



Detectio

n Binary Division in a CRC Checker



Detection(cont'd)

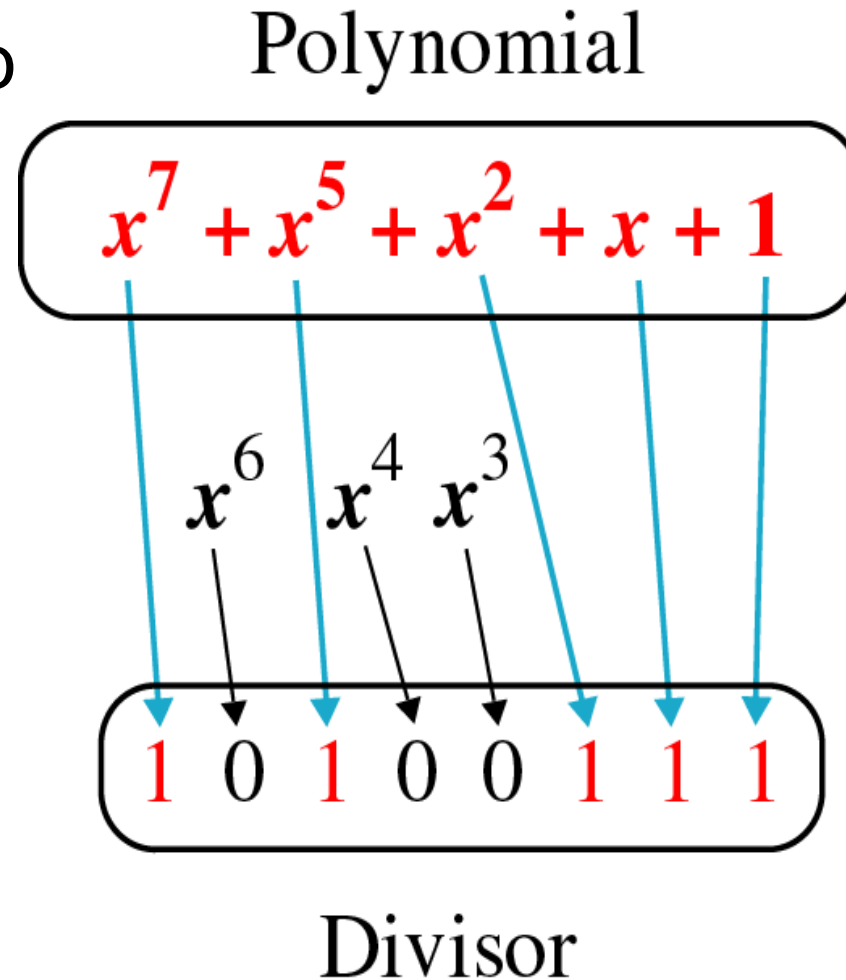
- Polynomials

- CRC generator(divisor) is most often represented not as a string of 1s and 0s, but as an algebraic

$$x^7 + x^5 + x^2 + x + 1$$

Detection(cont'd)

- A polyno
divisor



Detection(cont'd)

- Standard polynomials

CRC-12

$$x^{12} + x^{11} + x^3 + x + 1$$

CRC-16

$$x^{16} + x^{15} + x^2 + 1$$

CRC-ITU-T

$$x^{16} + x^{12} + x^5 + 1$$

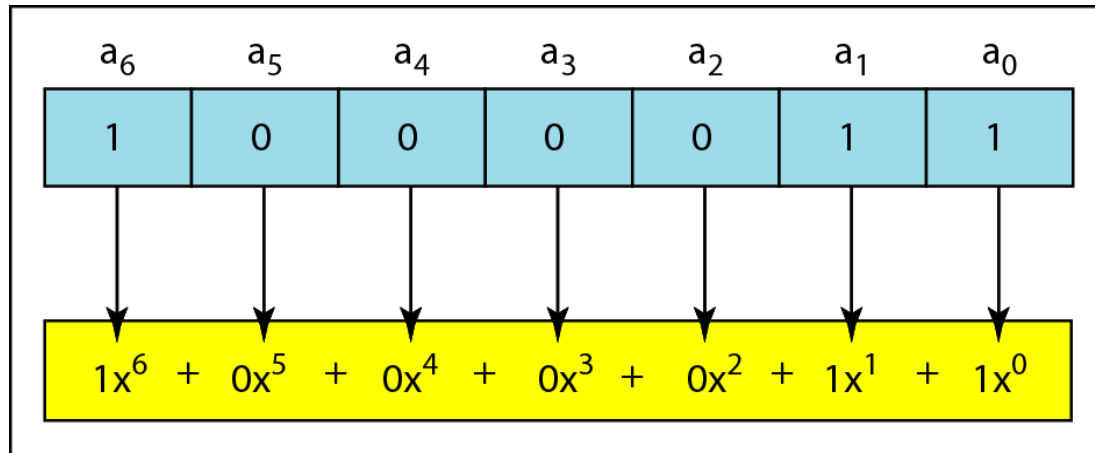
CRC-32

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

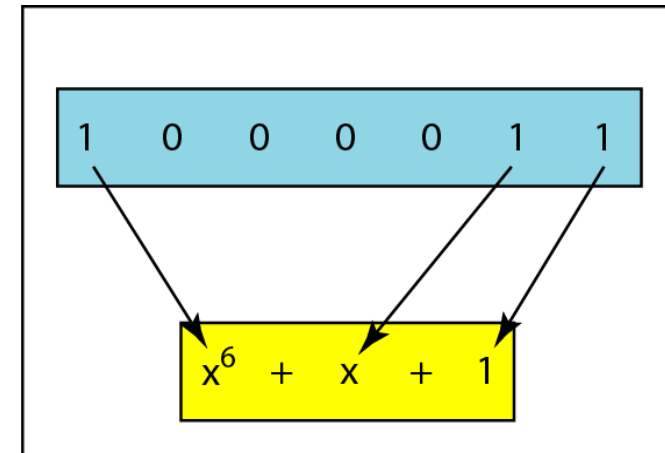
Using Polynomials

- We can use a polynomial to represent a binary word.
- Each bit from right to left is mapped onto a power term.
- The rightmost bit represents the “0” power term. The bit next to it the “1” power term, etc.
- If the bit is of value zero, the power term is deleted from the expression.

Figure 10.21 *A polynomial to represent a binary word*



a. Binary pattern and polynomial



b. Short form

Table 6-4 *Error detection performance of cyclic redundancy checksum*

Type of Error	Error Detection Performance
Single bit errors	100 percent
Double bit errors	100 percent, as long as the generating polynomial has at least three 1s (they all do)
Odd number of bits in error	100 percent, as long as the generating polynomial ends with a Factor of + 1) (they all do)
An error burst of length $< r+1$	100 percent
An error burst of length $= r+1$	probability = $1 - (1/2)^{r-1}$
An error burst of length $> r+1$	probability = $1 - (1/2)^r$

Application

- When a codeword is received or read, a CRC-enabled device performs a CRC on the whole codeword and compares the resulting check value with an expected *residue* constant.
- If the CRC check values do not match, then the block contains a data error
- The device may take corrective action, such as rereading the block or requesting that it be sent again.

1. A bit stream 10011101 is transmitted using the standard CRC method. The generator polynomial is x^3+1 .

1. What is the actual bit string transmitted?

2. Suppose the third bit from the left is inverted during transmission. How will receiver detect this error?

2. Given the dataword 1010011110 and the divisor 10111, show the generation of the CRC codeword at the sender using binary division.

In a CRC error-detecting scheme, choose the generator polynomial as $x^4 + x + 1$ (10011). Encode the 11-bit sequence: 10010011011

or

Given a CRC generator $x^4 + x + 1$ (10011), calculate the CRC code for the message 10010011011.

Show the steps clearly and derive the solution

Consider a CRC code with a generator polynomial of $g(x) = x^4 + x^3 + 1$.

- a. (20 points) Show step by step (using the longhand division) how to find the codeword that corresponds to information bits of 1101.

Justify answer using CRC for divisor: 1101 dividend: 100100.

- **Example Question:**

- **? : A bit stream 10101110 is transmitted using CRC. X^3+1 is the generator polynomial used. Find out the codeword to be transmitted. Suppose during the transmission, due to some noise 5th bit from left of the code word flipped. Show how CRC detects the error.**

Hamming Code – Error Detection and Correction

In data communication, a **hamming code** is an **error-correcting code** named after its inventor, Richard Hamming.

Hamming codes can **detect single** and up to 3 bits error, and correct **single-bit error** as well.

Hamming Codes

- Hamming codes are error-correcting codes
- These codes were originally designed with $d_{min}=3$, which means that they can detect up to two errors or correct one single error.
- relationship between n and k in a Hamming code. We need to choose an integer $m \geq 3$. The values of n and k are then calculated from $n = 2^m - 1$ and $k = n - m$. The number of check bits $r = m$.
- For example, if $m = 3$, then $n = 7$ and $k = 4$. This is a Hamming code $C(7, 4)$ with $d_{min}=3$.

Steps

- Steps includes:
- 1) Calculate number of redundant bits.
- 2) Positioning the redundant bits.
- 3) Calculate the values of each redundant bits
- $2^r \geq d+r+1$; r: no: of redundant bits
- d: no: of data bits.
- Powers of two 1,2,4, 8...
- Even Parity / Odd Parity.

❖ Hamming Code Structure

- All the bit positions that are power of 2 are marked as parity bits (1,2,4,8,.....) and other bits are for data.

Eg: 7 bit hamming code

D7	D6	D5	P4	D3	P2	P1
----	----	----	----	----	----	----

Parity bit
values :

Eg: 1101

D7	1	D6	P4	D5	P2	P1
?	1	0	?	1	?	?

P1 : check 1 bit & skip 1 bit (1,3,5,7,9,.....)

P2 : check 2 bit & skip 2 bit (2,3,6,7,10,11,...)

P4 : check 4 bit & skip 4 bit (4,5,6,7,12,13,14,15,.....)

D7	D6	D5	P4	D3	P2	P1
1	1	0		1		

P1 D3 D5 → P1 → P1 =

D7 P2 D3 → 101 → 0

D6 D7 P4 → P2 → P2

D5 D6 D7 111 =1

P4 P4 =

1	1	0	011	0	01	1	0
---	---	---	-----	---	----	---	---

❖ **DETECTING & CORRECTING ERROR**

• Suppose the receiver receives the data **1011011**

1		0		1		1		0		1		1
---	--	---	--	---	--	---	--	---	--	---	--	---

- **Step 1:** analyze bit 1,3,5,7 (for P1) = P1 D3 D5 D7 = 1011 Odd number of 1s. So error exist and we put **P1=1**
- **Step 2:** analyze bit 2,3,6,7 (for P2) = P2 D3 D6 D7 = 1001 even number of 1s. So no error and we put **P2=0**
- **Step 3:** analyze bit 4,5,6,7 (for P4) = P4 D5 D6 D7 = 1101 Odd number of 1s. So error exist and we put **P4=1**

Correcting Error

- Error word E =

1	0	1
---	---	---

P2 P1
- Decimal value of the error $E = 5$ which shows that the 5th bit is in error.
- So we write the correct word by simply inverting only the 5th bit

1 0 0 1 0 1 1

- In a CRC error-detecting scheme, choose the generator polynomial as $x^4 + x + 1$ (10011). Encode the 11-bit sequence: 10010011011
- Given a CRC generator $x^4 + x + 1$ (10011), calculate the CRC code for the message 10010011011. Show the steps clearly and derive the solution.
- Justify answer using CRC for divisor: 1101 dividend: 100100
- The data bit to be transmitted is 1011010, to be solved using the hamming code method.
- Encode a binary word 11001 into the even parity hamming code.
- Let us assume the even parity hamming code 111001101 is transmitted and the received code is 110001101). Now from the received code, let us detect and correct the error.
- The message 11001001 is to be transmitted using CRC polynomial $x^4 + 1$ to protect it from errors. Find the message that should be transmitted .
- Construct the even parity Hamming code word for a data byte 1001101.